



Universidad
Indoamérica

UNIT 7 · FUNDAMENTALS OF OPERATING SYSTEMS

Threads and concurrency

Universidad Tecnológica Indoamérica

Lab TD6

A thread: a sequential execution inside a process

- Also a "lightweight process": several executions in one process
- They all share the **same address space**
- **There is no memory protection** between threads
- A process = virtual memory (protection) + threads (parallel execution)

What is shared, with real addresses

process · 8 thread(s)

code

data: shared

0x650ff476602c = 8

heap

descriptors 0 1 2

shared by every thread

thread 1

stack

local 0x31fee74

registers: PC, SP...

thread 2

stack

local 0x19fbe74

registers: PC, SP...

thread 3

stack

local 0xbffee74

registers: PC, SP...

thread 4

stack

local 0x29fde74

registers: PC, SP...

thread 5

stack

local 0x21fce74

registers: PC, SP...

thread 6

stack

local 0x11fae74

registers: PC, SP...

thread 7

stack

local 0x09f9e74

registers: PC, SP...

thread 8

stack

local 0xb7fde74

registers: PC, SP...

stacks 8.004 MiB apart (8 MiB + 4 KiB guard)

Multithreaded programs

- Embedded systems: one program, concurrent operations
- The kernel itself: thousands of kernel threads
- Databases and network servers (unit 8)
- Parallel programs: OpenMP, one thread per core

Today also

Browsers (threads per tab and per task), game engines, language-model inference on CPU (unit 9) and Python, which since version 3.13 offers a variant without the global lock (GIL).

Creating a thread is cheaper... by how much?

Real output

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

- No pages to copy: only a new stack and a task structure
- The system schedules each thread: **do not assume any execution order**

Creating, ending and waiting for a thread

C

```
int  pthread_create(pthread_t *t, const pthread_attr_t *attr, /* NULL: defaults */
                  void *(*start)(void *), void *arg);
void pthread_exit(void *retval); /* = return retval from start; never returns */
int  pthread_join(pthread_t t, void **retval); /* PTHREAD_CANCELED if t was cancelled */
int  pthread_detach(pthread_t t); /* nobody will join t: freed when it ends */
```

- `attr` chooses, for instance, *joinable* (default) or *detached*: it is set up with `pthread_attr_init` and `pthread_attr_setdetachstate`
- A thread that ends **does not free** its stack or its state until someone joins it (or if it is detached): otherwise, a memory leak, like a zombie
- `join` suspends the caller until `t` ends and collects its `retval`; a detached thread cannot be joined
- On Linux, `pthread_create` uses the `clone` call: same address space, same descriptors

pthread_create, pthread_exit, pthread_join

C tds/td6/threads.c

```
for (int i = 0; i < NTHREADS; i++) {  
    int rc = pthread_create(&tids[i], NULL, start, (void *)(&i));  
    exit_if(rc != 0, "pthread_create"); /* pthread functions return the error code */  
}  
for (int i = 0; i < NTHREADS; i++)  
    exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");
```

Real output

```
$ ./threads  
thread 57632 of process 357789: shared[0x650ff476602c] = 1, local[0x7626e31fee74] = 1  
thread 79520 of process 357789: shared[0x650ff476602c] = 4, local[0x7626e19fbe74] = 1  
...
```

Fixes (slides 137–140): `-pthread`, not `-lthread`; `pthread_*` returns 0 or the error number, **not** -1 with `errno`.

Threads and signals

Signals belong to **the whole process**. For threads:

PROCESS	THREADS
<code>kill()</code>	<code>pthread_kill()</code>
<code>sigprocmask()</code>	<code>pthread_sigmask()</code>
<code>sigsuspend()</code>	<code>sigwait()</code> (a thread dedicated to waiting for signals)

Concurrency and shared memory: headaches

Most of the time threads touch different data and the order does not matter. But with `x = 1` initially:

```
c
/* thread 1 */      /* thread 2 */
x = x + 2;           x = x * 3;
```

What values can `x` take? → **non-deterministic**.

- It holds equally for threads and for processes with shared memory (unit 6)
- Bugs that are hard to reproduce: they show up once every millions of runs
- A worse example (slide 146):
`while (x < 1000) x++;` against
`while (x > -1000) x--;` either may win... or it may never end

The 20 interleavings, one by one

thread 1: $x = x + 2$;

1. $r1 = x$

2. $r1 = r1 + 2$

6. $x = r1$

thread 2: $x = x * 3$;

3. $r2 = x$

4. $r2 = r2 * 3$

5. $x = r2$

memory

$x=1$

$x=1$

$x=1$

$x=1$

$x=3$

$x=3$

result of the 20 interleavings

$x = 3: 18$

$x = 9: 1$

$x = 5: 1$

real, TD6 (8 threads, 100,000 tickets):

tickets_race: 103,195 sold (tickets = -7)

tickets_mutex: 100,000 sold (tickets = 0)

tickets_sem: 100,000 sold (tickets = 0)

Atomic operations

- They run completely or not at all: **indivisible**
- Typically: a memory access of native size, a simple instruction
- Not atomic: `x++` (read-add-write), accessing 64 bits on a 32-bit machine

In modern C

Since C11, two threads accessing the same variable without synchronization (with at least one write) is a **data race: undefined behaviour**. For atomics: `_Atomic`, `<stdatomic.h>` or `__atomic_add_fetch` (used by `shm_fork`, unit 6). `volatile` makes nothing atomic.

Too much milk: mutual exclusion in software

THREAD 1	THREAD 2
looks in the fridge: no milk	
goes to the store	looks in the fridge: no milk
buys milk	goes to the store
returns	buys milk... too much!

- **Synchronization:** using atomic operations to coordinate threads
- **Mutual exclusion:** only one thread does the action at a time
- **Critical section:** the code only one thread may run at a time
- Solutions 1–3: locking the fridge, one note, one note per thread... and their failures

Peterson and Dekker: correct... on paper

C

```

/* Peterson, thread 1 */
postit_1 = true; turn = 2;
while (postit_2 && turn == 2)
    ;                               /* busy waiting */
if (!milk) milk++;                 /* critical section */
postit_1 = false;

```

- Correct if memory accesses are atomic and **ordered**
- But the compiler and the processor **reorder** (slide 154)
- And they busy-wait: they burn CPU

Real output

```

$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do ./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
...
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
...

```

Every solution, every state

thread 0

```

postit_0 = 1
turn = 1
/* no fence */
loop: q = postit_1
if (!q) goto cs
t = turn
if (t ≠ 1) goto cs
goto loop
cs: enter()
r = milk
if (r) goto x
milk++
x: leave()
postit_0 = 0
end

```

thread 1

```

postit_1 = 1
turn = 0
/* no fence */
loop: q = postit_0
if (!q) goto cs
t = turn
if (t ≠ 0) goto cs
goto loop
cs: enter()
r = milk
if (r) goto x
milk++
x: leave()
postit_1 = 0
end

```

shared memory

```

milk = 0
postit_0 = 0
postit_1 = 0
turn = 0

```

thread 0 buffer: p0=1, turn=1

thread 1 buffer: p1=1, turn=0

BOTH IN THE CRITICAL SECTION!

exhaustive exploration

ends with 1 litre(s): 14 final state(s)

ends with 2 litre(s): 4 final state(s)

shortest counter-example: 12 steps (step 12)

Atomic instructions: test-and-set and compare-and-swap

c

```
int test_and_set(int *p) {    /* atomic */
    int old = *p;  *p = 1;  return old;
}
/* spinlock: busy-waits until the old value is 0 */
void lock(int *l)    { while (test_and_set(l)) ; }
void unlock(int *l) { *l = 0; }
```

- x86: `xchg`, `lock cmpxchg`, `lock xadd`; ARM64: `ldxr` / `stxr` or `cas`
- Busy waiting wastes CPU and can cause **priority inversion**
- A lock without waiting? No. Without busy waiting? Partly →

Limited busy waiting, today: the futex

Real output

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e trace=futex ./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
  0.00    0.000000    0    1    futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00    0.223730    27    8013    3116 futex
```

- Free mutex → one atomic instruction in user space, **no system call**
- Busy mutex → `futex(WAIT)`: the kernel puts the thread to sleep and wakes it on `unlock`
- `pthread_spin_lock` never sleeps (busy waiting): only for very short sections where sleeping would cost more than spinning

Locks: `pthread_mutex_lock` / `unlock`

C tds/td6/tickets_mutex.c

```
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

...
for (;;) {
    pthread_mutex_lock(&lock);
    if (tickets ≤ 0) {                /* checked w.
        pthread_mutex_unlock(&lock);
        break;
    }
    sold++;
    sched_yield();
    tickets--;
    pthread_mutex_unlock(&lock);
}
```

Or dynamic: `pthread_mutex_init(&m, attr)`
 (`pthread_mutexattr_t`; `NULL` = defaults) ...
`pthread_mutex_destroy(&m)` .

Real output

```
$ ./tickets_race 100000 | tail -3
...
103195 tickets sold out of 100000 (tickets = -7)
```

Real output

```
$ ./tickets_mutex 100000 | tail -3
...
100000 tickets sold out of 100000 (tickets = 0)
```

Fix (slide 162): it returns 0 or the error number
 (`EDEADLK` only in error-checking mutexes), not
 -1.

Semaphores (Dijkstra, late 1960s)

An integer ≥ 0 with two atomic operations: **P** (*proberen*: waits until positive and subtracts 1) and **V** (*verhogen*: adds 1 and wakes up).

- Initial value 1 $\rightarrow$ lock
- Initial value 0 $\rightarrow$ waiting for another thread
- Initial value k $\rightarrow$ at most k at a time

C

```
int sem_init(sem_t *s, int pshared, unsigned value);
int sem_destroy(sem_t *s); /* unnamed */
sem_t *sem_open(const char *name, int flags /* O_CREAT [/ O_EXCL] */,
                mode_t mode, unsigned value); /* SEM_FAILED */
int sem_close(sem_t *s); /* named: this process */
int sem_unlink(const char *name); /* named: the name */
int sem_wait(sem_t *s); /* P */
int sem_post(sem_t *s); /* V */
```

Fix (slide 167): `sem_open` returns `SEM_FAILED` (not `-1`) and has no default value: with `O_CREAT`, `value` must be given. And `sem_close` does not delete the semaphore: the name stays in `/dev/shm` until `sem_unlink`.

The producer and consumers problem

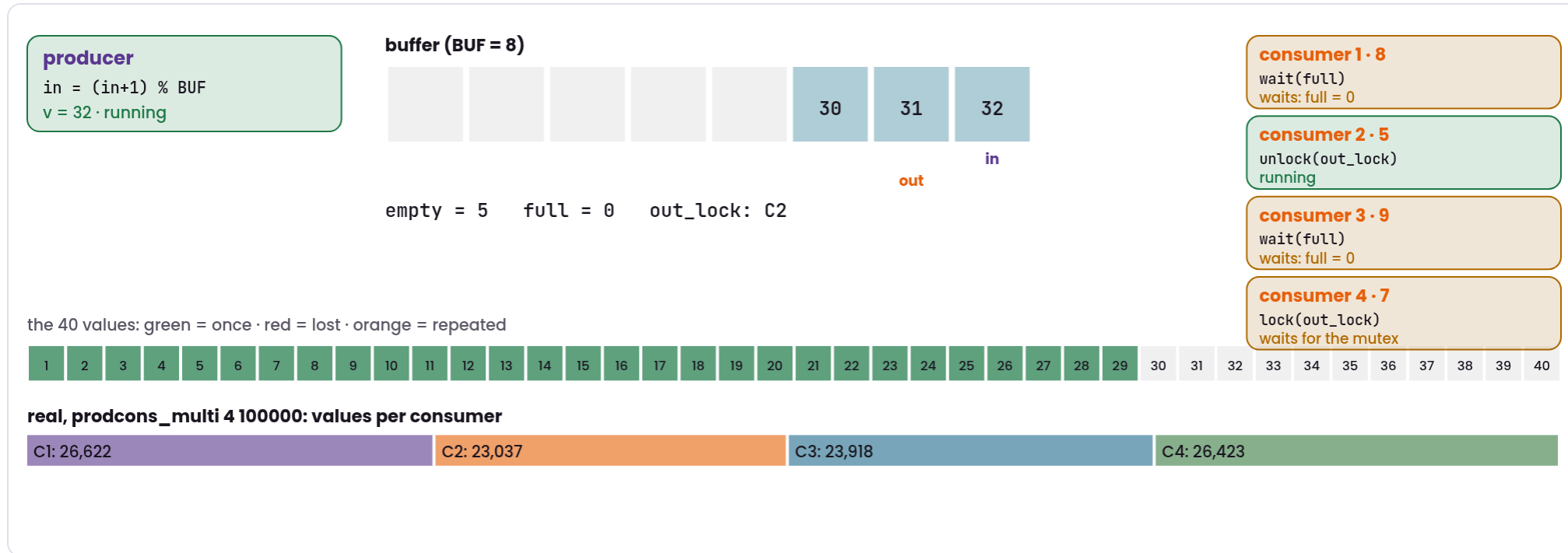
- A producer writes data into a shared buffer of limited size
- The consumers take them out
- The producer waits if the buffer is full; the consumer, if it is empty
- The buffer is modified: a critical section is needed

Examples: a command pipeline (unit 5),
`gcc ( cpp | cc1 | as | ld )`, image filters,
message queues between services.

Real output

```
$ ./prodcons_multi 4 100000  
consumer 2: 23918 items  
consumer 3: 26423 items  
consumer 1: 23037 items  
consumer 0: 26622 items  
sum = 5000050000, expected 5000050000: OK
```

Two semaphores and a mutex, step by step



Condition variables

C

```
pthread_mutex_lock(&mutex);
while (!condition)           /* while, not if */
    pthread_cond_wait(&cond, &mutex);
/* condition is true, mutex held */
pthread_mutex_unlock(&mutex);
/* whoever changes the condition: */
pthread_cond_signal(&cond);   /* wakes one */
pthread_cond_broadcast(&cond); /* wakes all */
```

- Wait for any condition (not just a counter)
- `cond_wait` releases the mutex and sleeps **atomically**, and takes it again on waking up
- `while` and not `if`: there may be spurious wakeups or another thread may have changed the condition

Deadlocks

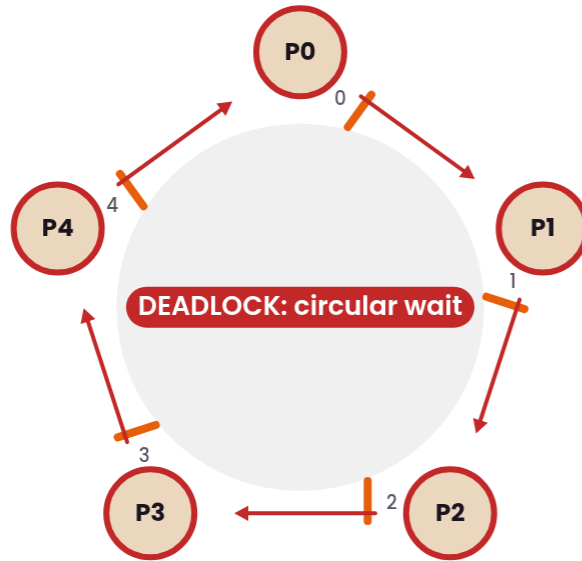
C

```
/* thread 1 */      /* thread 2 */
x.P();              y.P();
y.P();              x.P();    /* ← may never return */
/* ... */           /* ... */
y.V(); x.V();        x.V(); y.V();
```

It does not always happen: it depends on the execution. It needs more than one resource and it is not resolved without outside intervention.

- Resources: CPU, memory, disk, devices; shareable or not; preemptible or not
- Example: a crossing with right-of-way, each car holding a segment and waiting for the next
- Ways out: rolling back, taking a resource away (preemption)... with a risk of starvation

The dining philosophers



- thinks
- is hungry
- eats
- done
- waits for (graph)

Coffman conditions

- ✓ mutual exclusion
- ✓ hold and wait
- ✓ no preemption
- ✓ circular wait

The four conditions and how to prevent them

1. Mutual exclusion
 2. Holding resources while waiting for others
 3. No preemption
 4. Circular wait: T_1 waits for T_2 , ..., T_n waits for T_1
- **Global order** of acquisition (breaks 4): the TD6 fix
 - Preempting (breaks 3), with a risk of inconsistent state
 - Detecting cycles in the allocation graph; avoiding them with the banker's algorithm
 - Timeouts (`trylock`, `timedlock`): no guarantee

Conclusion

- Problems: non-deterministic results, deadlocks, starvation
- Every write to a shared resource → in a critical section
- Mutual exclusion in software works on paper; on real hardware, **mutexes, semaphores and atomics**
- Deadlocks are hard to reproduce and require all four conditions

Solved TD6 + AI challenge

- `threads`, `tickets_race/mutex/sem`
- `prodcons`, `prodcons_multi`, `philosophers` (and its deadlocking version)
- New: `create_cost`, `peterston`, `lock_cost`

Bash

```
make -C tds && tds/tests/run.sh td6
```



Universidad
Indoamérica

TAKE-AWAYS

Three questions

1. Why does `tickets_race` sell more tickets than there are?
2. Why does Peterson fail on an x86 and what fixes it?
3. Which condition does the global chopstick order break?